HYPERLOGS / DOCUMENTACIÓN TÉCNICA

Seguridad.
Con una mirada clara.

Arquitectura, tecnología y operación

Plataforma de bitácoras, detección, investigación y respuesta controlada. Incluye descubrimiento de activos en la red local.

Versión documentada	Fecha de edición
1.1.0 + descubrimiento LAN	28 de octubre de 2026

React · FastAPI · SQLite · Linux

Escenario: centro de cómputo y laboratorio SOC.

Estado: implementación local revisada; puesta en servicio Linux y fuentes del centro pendientes de validación.

CAPÍTULO 00

Guía de lectura

El documento combina fundamentos, diagramas, contrato técnico y manual operativo.

Capítulo	Contenido
01	Resumen, problema, objetivos y alcance
02	Requisitos y criterios de aceptación
03	Arquitectura de componentes y despliegue
04	Tecnologías y decisiones
05	Flujo del evento y límites de confianza
06	Modelo de datos y relaciones
07	Gobernanza y responsabilidades
08	Detalle de implementación y contratos
09	Instalación, operación y recuperación
10	Resultados de revisión y riesgos residuales
11	Trazabilidad con la práctica y evolución
12	API, estructura y referencias

La descripción se basa en el código y en las guías del proyecto. Los ejemplos de red son ilustrativos y deben reemplazarse por las redes autorizadas del centro. Las configuraciones de producción requieren revisar dominio, certificado, propietarios y rutas antes de habilitar el servicio.

La revisión diferencia capacidad implementada, prueba local y evidencia pendiente. No declara que una simulación sustituya una integración real ni que el inventario de red garantice encontrar todos los dispositivos conectados.

CAPÍTULO 01

Problema, objetivos y alcance

Una necesidad concreta: convertir registros dispersos en decisiones documentadas.

En un centro de cómputo los registros se distribuyen entre computadoras, aplicaciones y equipos de comunicaciones. Revisarlos por separado retrasa la identificación de accesos anómalos y cambios de configuración. También dificulta reconstruir quién investigó una alerta, qué evidencia utilizó y por qué se autorizó una respuesta.

HyperLogs concentra señales en un servidor y permite recorrer su ciclo: recopilación, detección, investigación, análisis histórico, respuesta y auditoría. Se plantea para una red de laboratorio y recursos limitados, con una interfaz accesible desde el navegador.

Objetivo general

Implementar una plataforma web de operación local que centralice eventos de fuentes identificadas, genere alertas mediante correlación y conserve expedientes, decisiones y resultados verificables con separación de responsabilidades.

Objetivos específicos

1. Centralizar eventos con origen, tiempo, severidad y procedencia.
2. Descubrir activos que responden en la LAN y facilitar su registro.
3. Detectar cuatro escenarios con umbrales reproducibles.
4. Conservar cronología, hipótesis y evidencias por incidente.
5. Estimar tendencias con un método estadístico explicable.
6. Ejecutar tres respuestas de laboratorio con autorización y recuperación.
7. Mantener permisos, bitácoras y reportes de auditoría.

Alcance y límites

La administración abarca operadores, fuentes, inventario, evidencias e investigación. Obtener métricas o registros de un dispositivo requiere su agente o una integración. Una IP descubierta no otorga permisos para modificar el host. No existe instalación silenciosa del agente ni administración remota universal.

Las respuestas suministradas actúan en el sandbox y no cambian el firewall del sistema operativo. El diseño supone una organización y una instancia; no implementa multicliente aislado, múltiples réplicas ni alta disponibilidad. La puesta en servicio del centro exige comprobar fuentes reales, TLS y restauración en su equipo Linux.

CAPÍTULO 02

Requisitos y aceptación

SRS resumida: comportamiento que debe poder demostrarse.

ID	Requisito	Criterio observable
RF01	Identidad y roles	Servidor valida sesión y permisos por operación.
RF02	Ingesta	Fuente autenticada, UTC, validación y deduplicación.
RF03	SIEM	R01-R04 producen alertas con referencias a eventos.
RF04	Investigación	Expediente con responsable, cronología y cierre documentado.
RF05	Análisis	Referencia histórica, anomalía y estimación justificadas.
RF06	SOAR	Tres acciones sandbox: solicitud, aprobación, efecto y reversión.
RF07	Auditoría	Actores y decisiones encadenadas, verificación y exportación.
RF08	Descubrimiento	Subred autorizada, progreso e inventario registrable.

Requisitos no funcionales

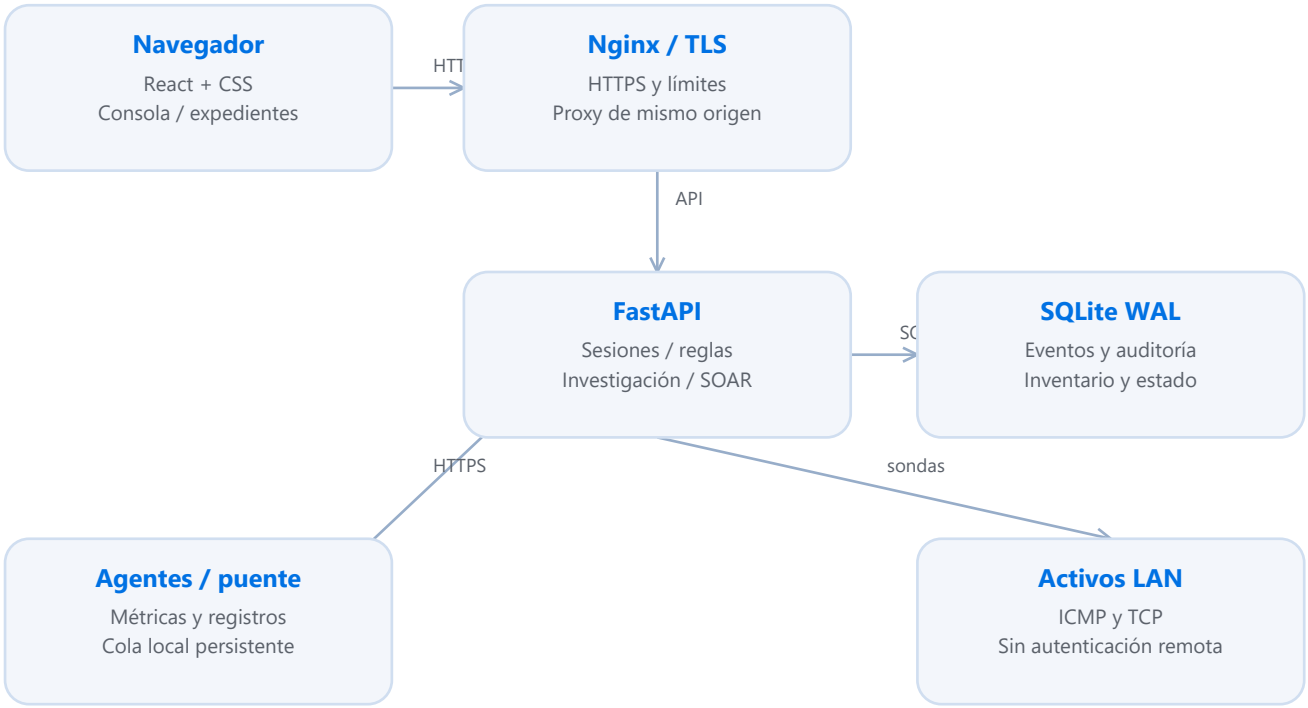
Seguridad: HTTPS, mínimo privilegio, sesión opaca, validación del servidor y credenciales por fuente. Usabilidad: mensajes comprensibles, formularios etiquetados, modales con foco controlado y tablas consultables en móvil. Rendimiento: consultas agrupadas, 3D diferido, límites de concurrencia y ejecución local. Recuperación: respaldo consistente y comprobación de integridad.

La práctica exige al menos tres fuentes, una de ellas real, cuatro reglas y tres playbooks. La aplicación soporta el recorrido y las pruebas cubren su lógica; la acreditación de fuentes reales requiere ejecutar el escenario en el centro y conservar capturas o exportaciones. El conjunto sintético inicial no acredita ese requisito por sí solo.

CAPÍTULO 03

Arquitectura de componentes

Visualización, procesamiento y recolección conectados por contratos explícitos.



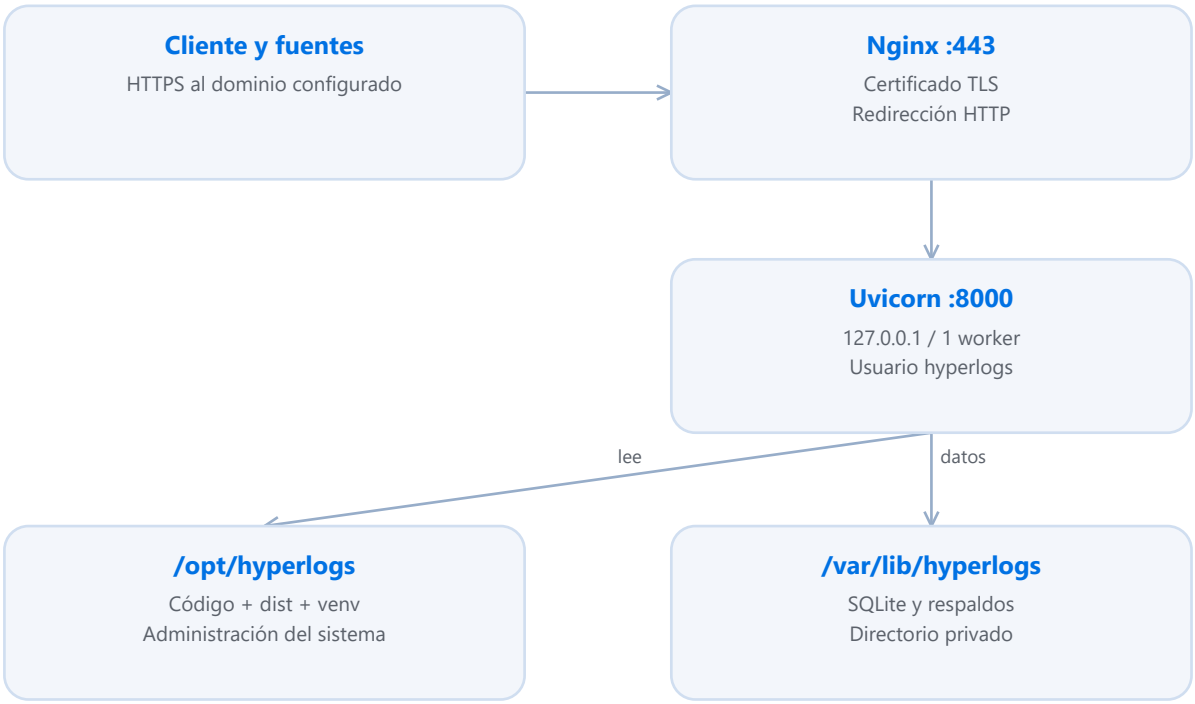
El navegador consulta la API del mismo origen y nunca se conecta a SQLite. Nginx termina TLS y reenvía a Uvicorn en loopback. El código React se compila en dist y se sirve como archivos estáticos; Node.js no necesita permanecer ejecutándose en producción.

FastAPI conserva autorización, validación, normalización, correlación y trazabilidad. SQLite opera con WAL y una coordinación local de escrituras. La recolección se realiza desde cada agente y se entrega por HTTPS con su propia credencial.

El descubrimiento usa la conectividad del servidor, no la del navegador remoto. La API responde al inicio con 202 y el trabajo publica progreso sin bloquear la navegación. Solo debe existir un proceso de API: agregar workers sin cambiar coordinación y persistencia introduciría carreras en acciones y trabajos.

Arquitectura de despliegue

Frontera de red, usuario del proceso y directorios.



El servicio systemd restringe privilegios y protege el sistema de archivos; únicamente necesita escribir en su directorio de datos. Los puertos de recepción pública corresponden al proxy, mientras que la API interna escucha en loopback. El puente Syslog recibe localmente y entrega por HTTPS.

El servidor necesita rutas y permisos de salida hacia las subredes autorizadas para descubrir dispositivos. Segmentación por VLAN, aislamiento inalámbrico o firewalls pueden limitar esa visibilidad. No se modifica la red del sistema operativo para evitar esas restricciones.

CAPÍTULO 04

Tecnologías y decisiones

Menos servicios permanentes para un equipo de recursos limitados.

Capa	Tecnología	Decisión
Interfaz	React 19 / Vite 6	Componentes y compilación estática.
Gráficos	Three.js / Lucide	Topología lógica e iconos consistentes.
API	Python / FastAPI / Pydantic	Modelos estrictos y autorización centralizada.
Ejecución	Uvicorn	ASGI en un proceso.
Datos	SQLite WAL	Sin servidor de base de datos separado.
Recolección	Python / psutil	Métricas y registros seleccionados del host.
Operación	Linux / Nginx / systemd	TLS, reinicio y usuario con permisos reducidos.
Calidad	pytest / npm audit / pip-audit	Flujos, permisos y dependencias.

SQLite fue una decisión adaptada al equipo disponible de aproximadamente 4 GB de RAM. Simplifica instalación y respaldo, aunque impone límites de concurrencia y coordinación. No es suficiente cambiar el número de workers para escalar: una evolución a PostgreSQL y una cola de trabajos requerirían revisar transacciones, bloqueos y distribución de tareas.

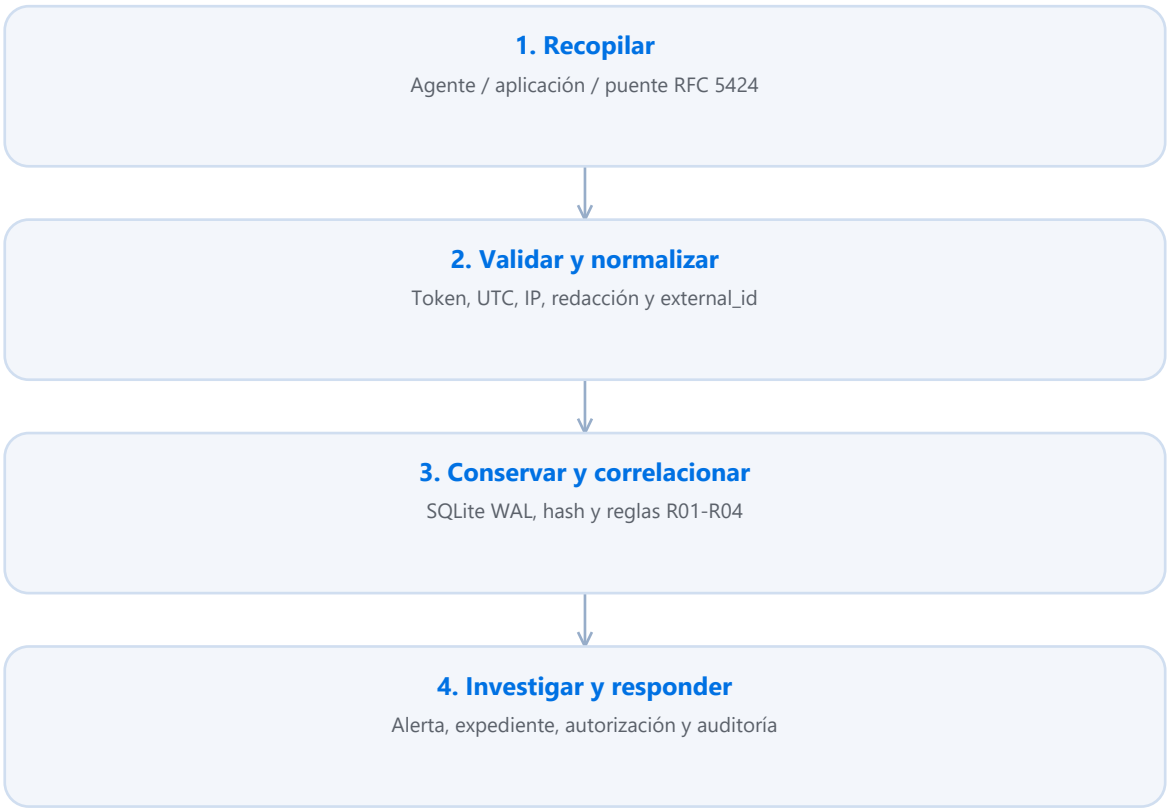
Las versiones runtime de backend quedan fijadas en requirements.lock y las de JavaScript en package-lock.json. npm ci reproduce la instalación frontend; pip instala el conjunto bloqueado. package.json establece Node 22.12 o posterior para desarrollo/compilación. No se agrega Pandas ni Scikit-learn porque la analítica implementada usa estadística básica de Python y SQL agregado.

La elección de Linux y SQLite reduce el número de servicios y el trabajo de administración. La capacidad real sigue dependiendo del volumen, el disco, los agentes, la retención y la configuración; las mediciones locales no garantizan rendimiento del equipo del centro.

CAPÍTULO 05

Flujo del evento y confianza

De la observación del host a una respuesta autorizada.



La identidad del emisor se obtiene al validar X-Source-Token. El agente no establece source_id, simulated ni risk: la API deriva esos campos de la fuente registrada y de sus reglas. Cada registro conserva el momento del evento y el de recepción, para distinguir generación de entrega.

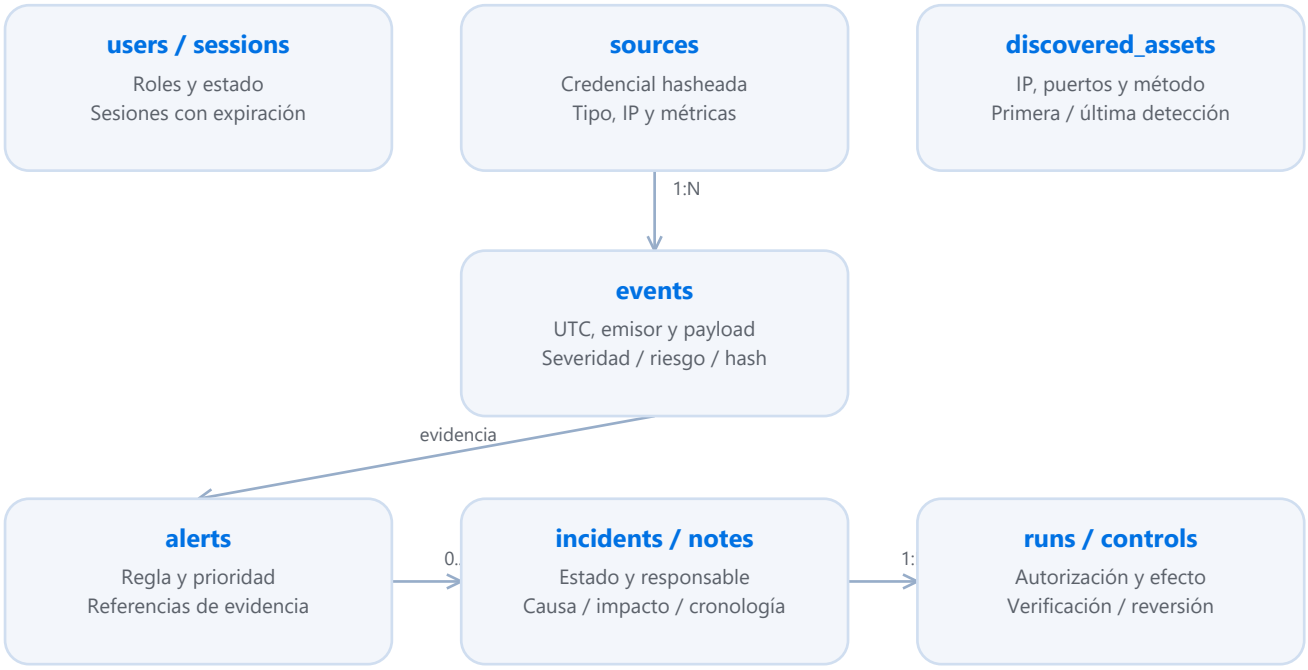
external_id debe ser estable cuando se retransmite y único por evento en una fuente. Una restricción por fuente evita duplicar datos al reintentar una entrega. La validación de IP y fecha evita contextos inválidos; no implica que el contenido declarado por una fuente comprometida sea verdadero.

Las alertas relacionan referencias de evidencia y las investigaciones añaden contexto humano. La correlación se ejecuta a la llegada; los históricos fuera de orden no se reprocesan globalmente. Esa limitación debe considerarse al importar registros atrasados.

CAPÍTULO 06

Modelo de datos

Relaciones entre operadores, señales, investigación y decisiones.



Events referencia sources y conserva el material normalizado. Alerts contiene contexto de regla y referencias a eventos. Incidents es único por alerta; notes añade observaciones con tiempo y actor. Runs y controls conservan solicitud, autorización, ejecución y efecto lógico de la respuesta. La tabla audit registra las decisiones transversales encadenadas con SHA-256.

Discovered_assets conserva observaciones por dirección IP. Se relaciona con una fuente real por coincidencia de IP al consultar el inventario. No es una identificación física permanente: una dirección puede cambiar de propietario por DHCP. Se recomiendan reservas y verificación para un vínculo duradero.

Las tablas e índices se crean o amplían al iniciar. No existe una herramienta independiente de migraciones versionadas. Es una mejora futura para controlar evoluciones complejas de esquema. Las escrituras se coordinan con un lock local, por lo que se conserva el modelo de un proceso.

Índices: events_time acelera búsquedas por fecha; events_correlation relaciona fuente, IP, clase y ventana; event_dedup evita duplicados por fuente y external_id; alert_source_status apoya el conteo de alertas por equipo y estado.

CAPÍTULO 07

Gobernanza y responsabilidades

Separación de funciones aplicada en cada operación del servidor.

Rol	Responsabilidad
Administrador TI	Usuarios, fuentes, tokens, descubrimiento e investigación; puede solicitar respuestas.
SOC nivel 1	Monitoreo y apertura de expedientes.
SOC nivel 2	Investigación, solicitud, ejecución y reversión autorizada.
CISO	Evaluación y aprobación o rechazo; no ejecuta respuestas.
Auditor	Consulta y exportación sin modificar evidencias.

Ocultar un botón no sustituye autorización. La API verifica el rol para cada escritura sensible. Una aprobación requiere una cuenta CISO distinta de quien solicitó. El cambio de rol o la desactivación revocan sesiones. Se evita la autodesactivación del administrador y se conserva al menos uno activo.

Confidencialidad: recolectar solo lo necesario, no enviar secretos y restringir acceso a bases y copias. Integridad: conservar eventos, decisiones y respaldos verificables. Escalamiento: SOC 1 identifica, SOC 2 investiga y CISO decide acciones de impacto. Reversión: preservar contexto, verificar efecto y dejar evidencia del resultado.

La retención organizacional no está automatizada. Deben fijarse responsables, tiempo de conservación, archivo de incidentes y copias externas. El producto aporta controles y trazabilidad, pero la política del centro debe aprobar esos plazos y procedimientos.

Caso de uso completo

Un agente entrega cinco fallos desde la misma IP dentro de dos minutos. R01 genera una alerta. SOC 1 valida contexto y abre un incidente. SOC 2 documenta hipótesis y solicita PB01. Otra cuenta CISO revisa y aprueba. SOC 2 ejecuta, comprueba el bloqueo lógico y registra el resultado. Antes del cierre se describen causa probable, impacto y riesgo residual. La reversión o expiración y las decisiones quedan en auditoría.

CAPÍTULO 08

Implementación y contratos

Detalle técnico de fuentes, SIEM, análisis, seguridad y descubrimiento.

Arquitectura, datos y políticas

Flujo

Agente / aplicación / puente RFC 5424 → HTTPS `POST /api/ingest` → validación de fuente → normalización y redacción → SQLite WAL → reglas → alertas → expedientes → solicitud SOAR → CISO → ejecución/verificación/reversión del sandbox → auditoría.

El navegador recibe solo datos y una prueba CSRF en memoria. La sesión opaca reside en una cookie HttpOnly; el servidor almacena su SHA-256 y expiración. Expira a las 8 horas o al reinicio del servidor. Logout, cambio/restablecimiento de contraseña y cambios administrativos revocan sesiones.

Modelo

• `users`, `sessions`: identidad, rol, estado activo y sesión.

• `sources`: tipo, procedencia, hash de token y última muestra.

• `events`: UTC, emisor, IP, usuario, clasificación, severidad original, riesgo, payload normalizado, SHA-256 y external_id opcional.

• `alerts`: regla, origen, prioridad y referencias a hasta 200 eventos recientes.

• `incidents`, `notes`: responsable, estado, causa, impacto, riesgo residual y cronología.

• `runs`, `controls`: solicitud, autorización, resultado y reversión privada. El contenido para revertir no se exporta ni se entrega al navegador.

• `audit`: decisiones y acciones encadenadas mediante SHA-256.

El riesgo es una puntuación de reglas, no una probabilidad de ataque. Syslog conserva 0–7 por separado. Eventos críticos por severidad tienen riesgo base 70, otros 15; cambios de configuración se elevan a 85. Las simulaciones usan 85. Las alertas se contextualizan por regla, fuente y origen.

Ingesta

```
{
  "events": [{
    "external_id": "host-log-record-123",
    "time": "2026-10-27T18:30:00-06:00",
    "kind": "login_failed",
    "user": "cuenta-de-prueba",
    "src_ip": "192.168.10.20",
    "dst_ip": "192.168.10.10",
    "category": "authentication",
    "code": "4625",
    "description": "Intento de autenticación fallido",
    "syslog_severity": 4,

    "metrics": {"cpu": 18, "memory": 36, "disk": 22}
  }]
}
```

Hasta 200 eventos por solicitud, cuerpo máximo 1 MiB. Las IP se validan, las fechas deben indicar zona y no exceder 5 minutos en el futuro. `external_id` deduplica por fuente. Su valor debe permanecer estable en una retransmisión y ser único por evento real. Los agentes no pueden establecer `source_id`, `risk` ni `simulated`; la procedencia se toma del registro de fuente. El campo `authorized` no suprime cambios reportados por fuentes: en esta versión se investiga conservadoramente.

Solo cpu/memory/disk, finitos entre 0 y 100. El receptor conserva la última muestra no vacía. Patrones conocidos de contraseña/token/Bearer se redactan antes de guardar/hash; esto es defensa adicional, no permiso para enviar secretos. Filtra esos datos en la fuente.

Reglas SIEM

ID	Condición	Alcance
R01	5 fallos en 120 s	Misma fuente y origen
R02	Login exitoso fuera de 07:00–20:00	Horario fijo UTC−6 del laboratorio
R03	Configuración modificada	Se presume no autorizada hasta investigar
R04	3 denegaciones seguidas de acceso administrativo en 300 s	Misma fuente y origen

Los lotes se ordenan por fecha. La correlación se ejecuta a la llegada; no hace reprocesamiento global de históricos fuera de orden. La política de horario corresponde al laboratorio y debe configurarse en código si tu organización opera en otra zona. Una alerta abierta acumula referencias recientes; los eventos originales no se modifican. No se implementa aprendizaje automático ni se atribuyen porcentajes de probabilidad.

Analítica

Una consulta agrupada obtiene volumen/riesgo por día UTC. Promedio y desviación de días completos; estimación del día siguiente con promedio móvil de siete días. Señala un aumento si supera media + 3σ, con σ mínimo 1 y al menos siete días completos con datos. El día actual es parcial y los datos sintéticos están contabilizados explícitamente. La estimación no se presenta si no hay línea base suficiente.

SOAR suministrado

PB01: bloqueo lógico de IP durante cinco minutos. PB02: restauración atómica del archivo `data/lab/service.conf` a su referencia. PB03: aislamiento lógico de la fuente en el sandbox. Se requieren solicitud de admin/SOC 2 y autorización de otra cuenta CISO. Ejecución y reversión quedan registradas; los fallos también. Una reversión de archivo no sobrescribe modificaciones posteriores. El sandbox no ejecuta comandos de shell ni modifica cuentas, procesos o firewall del host. `GET /api/lab/access` permite inspeccionar el control lógico; no protege por sí mismo otros servicios externos.

Producción desactiva estas acciones por defecto. Una plataforma en producción puede conservar SIEM, investigación, analítica y auditoría sin habilitar contención de laboratorio.

Operación e integridad

Un proceso API. Conexiones SQLite con foreign_keys y busy_timeout; WAL. Serialización de escrituras/acciones con lock local. Reportes en streaming mediante otra transacción de lectura; hasta 100 000 filas por tabla, con metadatos de truncación. CSV escapa fórmulas. Las exportaciones se auditan y nunca incluyen hashes de contraseñas, hashes de tokens ni material de reversión.

La verificación de auditoría recorre la cadena, no la carga completa en memoria. En instalaciones grandes, su costo crece con la historia; archiva y ancla periódicamente reportes fuera del servidor. La aplicación no purga automáticamente registros vinculados a evidencia. Define archivo y retención antes de crecer.

Cliente y rendimiento

React carga la escena 3D de forma diferida. Su render limita aproximadamente a 30 fps, respeta movimiento reducido, pausa fuera de pantalla/pestaña y reutiliza sombras estáticas. Las etiquetas solo actualizan React cuando cambian sus posiciones. Sin dependencias de fuentes o fotografías externas. La API agrupa históricos y cuenta fuentes sin consultas por equipo. El cliente evita consultas idénticas simultáneas, cancela búsquedas antiguas y muestra desconexión/expiración.

El UI conserva la estética aprobada. Formularios con etiquetas, diálogos con foco atrapado y restaurado, Escape, tablas navegables por teclado y mensajes de error. Las tablas amplias tienen desplazamiento horizontal en móvil; no se afirma conformidad WCAG integral sin una auditoría independiente.

Descubrimiento de activos en la LAN

La consola incluye Descubrir red. Un administrador inicia una búsqueda IPv4 de hasta 254 direcciones, en segundo plano, con progreso y auditoría. El servidor prueba ICMP y conexiones TCP a 22, 80, 443, 445 y 3389; no autentica, no lee banners ni ejecuta órdenes remotas. Las respuestas se conservan como inventario separado de las fuentes que envían eventos. Registrar abre el formulario de fuente con IP y tipo sugerido; el token y el agente completan la incorporación al monitoreo.

En el archivo de entorno del servidor define, con la red real del centro:

```
HYPERLOGS_DISCOVERY_NETWORKS=192.168.2.0/24
```

Puedes autorizar varias redes privadas separadas por comas. Solo se permiten bloques RFC1918 /16 o más pequeños; cada trabajo debe elegir una subred /24 a /30 contenida en la lista. Reinicia el servicio después de cambiarla. La instancia de desarrollo de esta computadora usa su Wi-Fi 192.168.2.0/24; esa dirección no debe asumirse en otro lugar. No se configura una red real por defecto en el paquete de producción.

Solo hay un trabajo simultáneo y un minuto mínimo entre inicios. Hasta 16 sondas se ejecutan en paralelo con tiempos de espera acotados. No requiere root; instala iputils-ping en Debian si falta ping. Si ICMP falla, se conserva la detección mediante los puertos TCP indicados. El servidor debe tener ruta y permisos de salida hacia los equipos. No expongas esa función a personas ajenas a la administración de la LAN.

La lista conserva primera y última detección; no afirma presencia actual. Un equipo que bloquea ICMP y los puertos probados puede no aparecer. VLAN, redes de invitados, aislamiento Wi-Fi y firewalls también limitan la visibilidad. No se identifica de forma confiable fabricante, MAC o sistema operativo; el tipo es una sugerencia editable. Una IP puede reasignarse por DHCP: para identidad duradera configura reservas y vincula agentes.

API: GET /api/discovery (inventario autenticado); POST /api/discovery/scan con {"network":"192.168.2.0/24"} (administrador y CSRF). Resultado 202; el cliente consulta el progreso cada dos segundos mientras el panel está abierto. Cerrar el panel no cancela el trabajo. Reiniciar el servidor interrumpe el trabajo activo pero conserva los activos ya guardados. El inventario queda en la misma base SQLite y se incluye en su respaldo.

CAPÍTULO 09

Instalación y operación Linux

Guía reproducible, incluyendo certificados, recuperación e inventario LAN.

Despliegue de una instancia en Debian/Linux

Usa un equipo dedicado o VM con Linux sin escritorio. La propuesta evita Docker, servicios de base de datos separados y compilaciones continuas en el servidor. Un solo proceso de API mantiene las escrituras SQLite y la cadena de auditoría coordinadas. Escalar a múltiples procesos requiere rediseñar esa coordinación; no aumentes `--workers` sin hacerlo.

1. Preparar archivos y usuario de servicio

Con permisos administrativos en Linux, instala Python 3.12+ con venv y Nginx desde los repositorios mantenidos de tu distribución. Copia el proyecto y el `dist/` compilado a `/opt/hyperlogs`. El paquete de entrega no incluye base de datos, usuarios, tokens, dependencias de desarrollo ni node_modules.

```
sudo useradd --system --home /var/lib/hyperlogs --shell /usr/sbin/nologin hyperlogs
sudo install -d -o hyperlogs -g hyperlogs -m 0700 /var/lib/hyperlogs
sudo install -d -o root -g hyperlogs -m 0750 /etc/hyperlogs
sudo python3 -m venv /opt/hyperlogs/.venv
sudo /opt/hyperlogs/.venv/bin/python -m pip install -r /opt/hyperlogs/requirements.txt
sudo chown -R root:root /opt/hyperlogs
sudo chmod -R a+rX /opt/hyperlogs
```

El código debe ser de solo lectura para el usuario del servicio. La base y los respaldos viven en `/var/lib/hyperlogs`, nunca dentro de `dist/`.

2. Configurar entorno

Copia `deploy/hyperlogs.env.example` a `/etc/hyperlogs/hyperlogs.env`, cambia el dominio e indica:

```
HYPERLOGS_ENV=production
HYPERLOGS_DEMO=0
HYPERLOGS_LAB_ENABLED=0
HYPERLOGS_DATA=/var/lib/hyperlogs
HYPERLOGS_PUBLIC_URL=https://tu-dominio.example
HYPERLOGS_ALLOWED_HOSTS=tu-dominio.example,localhost,127.0.0.1
```

No utilices comodines en hosts ni copies la base demo. Da al archivo permisos `root:hyperlogs`, modo `0640`. La URL pública es el origen exacto; no incluye rutas. Esta versión sirve frontend/API desde el mismo origen y no habilita CORS.

3. Bootstrap sin secretos en historial

Desde `/opt/hyperlogs`, ejecuta como usuario del servicio, configurando las mismas variables no secretas del archivo anterior. Por ejemplo:

```
sudo -u hyperlogs env HYPERLOGS_ENV=production HYPERLOGS_DEMO=0 \
 HYPERLOGS_DATA=/var/lib/hyperlogs \
 HYPERLOGS_PUBLIC_URL=https://tu-dominio.example \
 HYPERLOGS_ALLOWED_HOSTS=tu-dominio.example,localhost,127.0.0.1 \
 /opt/hyperlogs/.venv/bin/python -m backend.manage bootstrap
```

La contraseña se solicita con getpass. No se guarda en `.env`, scripts, reportes ni argumentos de shell. El usuario inicial es `admin`. Después crea cuentas individuales, especialmente la de CISO; no compartas la cuenta de administrador.

Recuperación de cuenta por un administrador del servidor: mismo entorno y `python -m backend.manage set-password nombre`. Esta operación revoca todas las sesiones de esa cuenta y deja auditoría.

4. HTTPS, proxy y servicio

Edita dominio y rutas de certificado en `deploy/nginx.conf`. Debes disponer de un certificado válido para ese dominio. Coloca `limit_req_zone` en el contexto `http{}` de Nginx y los bloques `server{}` en su configuración de sitio. Comprueba `sudo nginx -t` antes de recargar. El servidor interno escucha solo en `127.0.0.1:8000`; no publiques ese puerto. Nginx reemplaza X-Forwarded-For con la IP del cliente y Uvicorn confía únicamente en loopback.

```
sudo install -m 0644 /opt/hyperlogs/deploy/hyperlogs.service /etc/systemd/system/
sudo systemctl daemon-reload
sudo systemctl enable --now hyperlogs
sudo systemctl status hyperlogs
curl --fail https://tu-dominio.example/api/health
```

Permite solo los accesos de red necesarios a SSH/HTTPS y los colectores que hayas configurado. El ejemplo Nginx agrega compresión y rate limit de login; las cookies de producción tienen Secure, HttpOnly y SameSite=Strict. No se usa modo reload. Los procesos tienen límites de concurrencia/memoria y permisos reducidos.

5. Respaldar y restaurar

Instala `hyperlogs-backup.service` y `.timer`, recarga systemd y activa `hyperlogs-backup.timer`. El horario se define en UTC y el nombre contiene fecha UTC; el comando no imprime contraseñas. Archivos con permisos `0600` en el directorio privado del servicio.

El respaldo usa SQLite Backup API para incorporar cambios confirmados del WAL; no copies solo la base abierta. Conserva además una copia fuera de ese equipo, con cifrado y retención definidos por tu organización. El timer local no implementa esa copia externa ni borrado automático.

Para restaurar una copia propia y confiable: detén el servicio, extrae el archivo en un directorio de revisión, verifica el SHA-256 de la base contra `manifest.json`, prueba `PRAGMA integrity_check`, preserva la instalación anterior y reemplaza la base y `lab/` con el servicio detenido. No reutilices `-wal/-shm` de otra base. Ajusta propiedad/permisos y reinicia. El inicio invalida sesiones anteriores. Los tokens de fuente del respaldo corresponden a su fecha; rota los que se hayan comprometido o cambiado desde entonces.

6. Antes de permitir usuarios reales

• Verifica login/logout, cookie Secure, recarga de sesión y protección CSRF sobre tu HTTPS real.

- Comprueba que demo, Swagger y acciones de laboratorio están desactivados.
- Registra al menos una fuente real y verifica fecha, procedencia, cola y alertas.
- Comprueba un respaldo y restauración en un entorno separado.
- Mide volumen de eventos, tiempo de consulta y espacio de disco con tu carga; 4 GB de RAM no garantizan cualquier volumen.
- Vigila disco y errores; define archivo/retención. No se borran evidencias automáticamente.

Límites honestos

La API/web puede operar como un SOC pequeño de una instancia. No es una certificación, una prueba de penetración independiente ni un reemplazo completo de un SIEM empresarial. No se ha verificado systemd/Nginx/TLS en un host Linux real durante esta revisión. No incluye MFA, SSO, alta disponibilidad ni contención real de dispositivos. La cadena SHA-256 detecta alteraciones parciales; un administrador con control total del disco podría reconstruirla. Los backups/checkpoints externos son esenciales para anclar evidencias.

Habilitar descubrimiento LAN

Antes de iniciar el servicio, configura `HYPERLOGS_DISCOVERY_NETWORKS` en `/etc/hyperlogs/hyperlogs.env` con las redes privadas del centro, por ejemplo `192.168.2.0/24` si coincide con su red real. Puedes dejarlo vacío para desactivar escaneos. Instala `iputils-ping` desde los repositorios de Debian y verifica conectividad desde la cuenta `hyperlogs`. Reinicia `hyperlogs.service`. Desde Descubrir red, el administrador inicia el escaneo y registra los equipos encontrados. La instalación de agentes y sus permisos siguen siendo necesarios. Consulta TECNICA.md para restricciones, puertos y tratamiento de DHCP.

CAPÍTULO 10

Revisión y evidencia de calidad

Resultados locales, correcciones y alcance de la verificación.

Revisión de HyperLogs 1.1.0

Fecha: 27 de octubre de 2026. Alcance: código React/FastAPI, SQLite, agentes, interfaz y configuración de despliegue. Revisión local; no constituye certificación ni prueba de penetración externa.

Resultado

Se corrigieron los hallazgos siguientes y se preparó un despliegue de una instancia Linux mediante Nginx, HTTPS y systemd. No se ha desplegado ni validado el servicio en el equipo Linux del centro. Las automatizaciones actúan exclusivamente en el sandbox; no bloquean un firewall ni aíslan físicamente otras computadoras.

Seguridad: hallazgos corregidos

ID	Prioridad	Hallazgo y corrección	Evidencia
SEC-01	Alta	Sesiones expuestas al JavaScript: sustituidas por cookie HttpOnly, SameSite Strict, Secure en producción y prueba CSRF en memoria. Revocación por contraseña, desactivación o cambio de rol.	backend/app.py:196; src/api.js:1; pruebas de sesión, CSRF y revocación
SEC-02	Alta	Configuración de laboratorio utilizable por error: producción rechaza demostración, exige origen HTTPS y hosts explícitos; laboratorio desactivado por defecto. Sin contraseña predeterminada; alta local interactiva.	backend/security.py:19; backend/manage.py:13; prueba de proceso de producción aislado
SEC-03	Alta	Acciones sin límites suficientemente explícitos: comprobación de rol en servidor, solicitud/aprobación separadas, operaciones limitadas al sandbox y rechazo de reversión sobre cambios posteriores. Contenido privado de recuperación excluido de las respuestas.	backend/app.py; pruebas de los tres ciclos de playbook, fallo parcial y conflicto de recuperación
SEC-04	Alta	Fuentes y entradas no confiables: credenciales por fuente con rotación/revocación, procedencia determinada en servidor, validación de IP/fechas/métricas, deduplicación y redacción de secretos reconocibles.	backend/app.py:449; backend/security.py:41; pruebas de ingestión y tokens
SEC-05	Media	Frontera HTTP insuficiente: hosts/origen, CSP, tipos JSON, límite de cuerpo y tiempo, cabeceras y rate limit de acceso. Contraseñas PBKDF2-SHA256 de 600000 iteraciones y actualización de hashes anteriores.	backend/security.py:52; backend/app.py:57; deploy/nginx.conf
SEC-06	Media	Exportación y respaldo: neutralización de fórmulas CSV, consultas parametrizadas, límites declarados y respaldo consistente mediante SQLite Backup API con manifiesto SHA-256 y permisos privados.	backend/manage.py:48; tests/test_operations.py

La redacción reconoce patrones comunes: no garantiza eliminar cualquier dato sensible de un log. La cadena hash permite detectar alteraciones internas; un administrador con escritura total puede reescribir la cadena. Para mayor garantía se requiere una copia externa inmutable o anclaje externo.

Diseño, errores y fluidez

Se recorrieron login, consola, búsqueda, alertas, expediente, analítica, SOAR, auditoría, configuración y cuenta en navegador. Se verificó login y modal a 390 px sin desbordamiento horizontal. Se conservaron el diseño blanco, jerarquía visual y animación ambiental aprobados.

Correcciones: errores de conexión visibles con reintento; expiración vuelve a login; solicitudes duplicadas agrupadas y búsqueda anterior cancelada; protección frente a respuestas antiguas al cambiar sesión/rol; modales con foco, Escape y retorno de foco, montados fuera de la consola para respetar el tamaño de pantalla móvil; controles según permisos; gráficos sin series ficticias y baseline insuficiente señalado.

La escena 3D se carga después de autenticar, limita aproximadamente a 30 fps y pausa fuera de pantalla o pestaña inactiva; respeta movimiento reducido y reutiliza sombras. Se sustituyó una constante obsoleta de sombras de Three.js y se asegura repintado tras redimensionar. JavaScript inicial: unos 315 kB, 93 kB gzip; escena diferida: unos 588 kB, 151 kB gzip. Vite advierte del tamaño de este chunk diferido; no impide compilar. El CSS conserva capas de iteraciones visuales y queda como deuda de mantenimiento, sin bloqueo funcional observado.

Verificación reproducible

• 47 pruebas pytest aprobadas tras la ampliación LAN; cero fallos. Incluyen roles, reglas R01–R04, tres playbooks, auditoría alterada, exportación, backup WAL, agentes y configuración de producción aislada.

• Compilación de producción Vite correcta.

• npm audit: cero vulnerabilidades conocidas en el momento de la consulta.

• pip-audit: cero vulnerabilidades conocidas en las 13 dependencias runtime fijadas.

• Dos avisos de obsolescencia del entorno de pruebas (integración AnyIO/pytest y TestClient/httpx); no son fallos de ejecución de producción.

• Evidencias de pruebas y auditorías: docs/validation/.

Medición local con 5000 eventos sintéticos, Windows y ASGI TestClient: medianas de lectura entre 15 y 33 ms; p95 entre 17 y 36 ms. La medición excluye red y TLS y no es una garantía de capacidad en Linux o con hardware de 4 GB. La recepción del lote tardó 1.278 segundos en esa prueba aislada.

Antes de prestar servicio real

Seguir PRODUCCION.md: configurar dominio y certificado válidos, usuario de servicio, directorio privado de datos, administrador propio y un solo worker. Ejecutar nginx -t, comprobar systemd/HTTPS y restaurar un respaldo de ensayo. Registrar y probar las fuentes reales con sus permisos de sistema; su colección Windows/Linux y el puente rsyslog no se han probado en los equipos del centro. La demostración local no representa esas conexiones.

Configurar retención de eventos y copias externas según el uso real; actualmente no hay purga automática ni política de retención organizacional. SQLite y la coordinación de auditoría están diseñados para una instancia con un proceso. No usar múltiples workers ni varias réplicas sin rediseñar esa coordinación. No se incluye MFA/SSO ni operación distribuida de alta disponibilidad.

CAPÍTULO 11

Trazabilidad y evolución

Relación con los criterios de la práctica y cierre pendiente.

Criterio	Implementación	Evidencia por completar
Gobernanza y seguridad	Roles, aprobación y auditoría.	Políticas y responsables del centro.
Captura e integración	Agentes Windows/Linux, API y Syslog.	Al menos 3 fuentes y 1 real en el centro.
Normalización	UTC, severidad/riesgo, hashes y contexto.	Exportaciones de eventos reales.
Dashboard	Indicadores y topología lógica.	Demostración del escenario presencial.
SIEM	Cuatro reglas con pruebas.	Repetir disparadores con evidencias.
Investigación	Expediente y cronología.	Causa, impacto y cierre del caso.
Análisis predictivo	Media móvil y umbral estadístico.	Explicar cantidad y procedencia de datos.
SOAR	Tres playbooks de sandbox.	Aprobación, verificación y reversión en vivo.
Pruebas y reporte	Resultados y documentación.	Capturas y observaciones del centro.

La tabla no asigna una calificación. Diferencia las capacidades implementadas de las evidencias que requieren el entorno académico. Las opciones de base de datos del documento original son orientativas; SQLite debe justificarse por recursos y alcance de una instancia.

Mejoras prioritarias: validar Linux y fuentes reales; establecer retención y copia externa; identificar activos de forma duradera; incorporar migraciones y modularizar backend/CSS; evaluar MFA/SSO si aumenta la exposición; validar analítica con series reales; diseñar conectores de contención real con autorización y recuperación.

Los conectores de respuesta futura deben definir alcance, credenciales, idempotencia y comprobación del sistema destino. La evolución a varias instancias requiere persistencia y coordinación distribuidas, no solo aumentar procesos de Uvicorn.

CAPÍTULO 12

API, estructura y referencias

Punto de entrada para mantenimiento e integraciones.

Rutas	Uso
/api/auth/login; /me; /logout; /password	Identidad, sesión y prueba CSRF.
/api/sources; /{id}/rotate; /revoke	Registro y ciclo de credenciales de fuentes.
POST /api/ingest	Contrato de entrega de eventos.
/api/events; /alerts; /rules	Exploración, filtros y detección.
/api/incidents; /api/alerts/{id}/incident	Expediente y apertura desde alerta.
/api/playbooks/{id}/request; /api/runs/{id}/...	Solicitud, aprobación, ejecución y reversión.
/api/analytics; /overview; /audit	Indicadores e historial auditable.
/api/reports/export?format=json o csv	Exportación con límites y metadatos.
/api/discovery; POST /api/discovery/scan	Inventario y trabajo LAN.
/api/health; /config	Salud y configuración pública limitada.

Las rutas exactas y sus métodos se definen en backend/app.py y backend/discovery.py. Las operaciones de escritura autenticadas por cookie requieren CSRF; la ingesta usa X-Source-Token. Los modelos rechazan campos extra. Un 202 significa trabajo aceptado, no resultado terminado. La documentación OpenAPI queda desactivada en producción.

Estructura y referencias

Archivo o carpeta	Contenido
backend/app.py	Dominio: reglas, investigación, analítica, sandbox y persistencia.
backend/security.py	Frontera HTTP, entorno y redacción.
backend/discovery.py	Validación de redes, sondas y trabajo en segundo plano.
backend/manage.py	Alta local, contraseñas, backup y verificación.
src/	Interfaz, login, consola, escena y descubrimiento.
agents/	Recolección, transporte persistente y puente Syslog.
deploy/	systemd, Nginx, entorno, rsyslog y respaldo.
tests/; docs/validation/	Casos reproducibles y resultados registrados.

Fuente primaria: implementación HyperLogs 1.1.0 y ampliación de descubrimiento LAN revisadas al 28 de octubre de 2026. Repositorio: https://github.com/AmCharlyy/HyperLogs. Este documento describe el código entregado; no es una comparación de versiones externas de frameworks.

Requisitos académicos: Práctica Unidad 2, Gestión de amenazas cibernéticas con SIEM, Syslog y SOAR, Universidad Politécnica de Pénjamo, Seguridad Informática, siete páginas. Sus criterios se usan como fuente de requisitos; no otorgan permisos de operación sobre redes.

Documentos internos: README.md, docs/TECNICA.md, PRODUCCION.md, REVISION.md y docs/validation/. Estos archivos y el código prevalecen para reproducir comandos y comprobar límites técnicos.

Glosario

SIEM: gestión y correlación de eventos de seguridad. SOAR: orquestación y respuesta mediante un flujo de autorización. RBAC: permisos según rol. WAL: registro de escritura anticipada de SQLite. CSRF: inducción de una operación desde otro sitio. CIDR: notación de una subred. RFC1918: espacios IPv4 privados. Activo detectado: equipo que respondió a las sondas. Fuente: emisor registrado con credencial para entregar eventos. Sandbox: entorno acotado donde se ejecutan las acciones de esta versión.